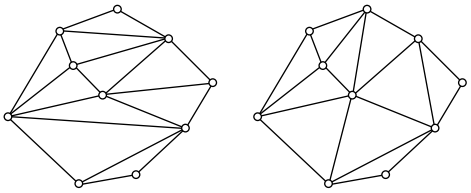
დელონეს ტრიანგულაცია და ვორონოის დიაგრამა
Delaunay Triangulation and Voronoi diagram

X.2

გამოთვლითი გეომეტრიის ერთ-ერთი ხშირი ამოცანაა მოცემული სიბრტყის, ან სიბრტყეთა სიმრავლის, დაყოფა სამკუთხედებად. ეს ამოცანა ფართოდ გამოიყენება გეოდეზიასა და კართოგრაფიაში, ტექნიკაში (განსაკუთრებით, სასრული ელემენტების მეთოდში).

მოცემული წერტილების სიმრავლეში უნდა მოხდეს მათი ერთმანეთთან დაკავშირება ისე, რომ მივიღოთ სამკუთხედები, რომელთა წვეროები მოცემული წერტილებია, და ამ სამკუთხედების წიბოები არ კვეთენ ერთმანეთს.

ერთ და იგივე წერტილების სიმრავლეზე შეიძლება სხვადასხვანაირი ტრიანგულაციის აგება (სურ....)



ტრიანგულაციის მეთოდის შერჩევა ხდება გარკვეული პარამეტრის საფუძველზე, მაგალითად:

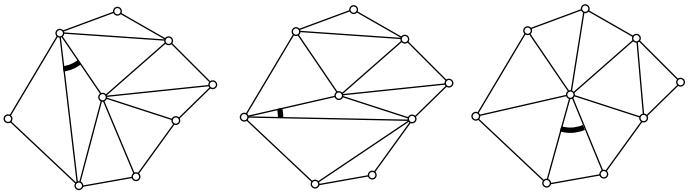
1. **მინიმალური წონის ტრიანგულაცია (minimum-weight triangulation)** - ისეთი ტრიანგულაცია, სადაც სამკუთხედების პერიმეტრების სიგრძეების ჯამი მინიმალურია.

2. **მინიმალური მაქსიმალური კუთხის ტრიანგულაცია (minmax angle triangulation)**: სამკუთხედების ყველაზე მცირე კუთხის მნიშვნელობა სხვა ტრიანგულაციებთან შედარებით მაქსიმალურია.

3. **მაქსიმალური მინიმალური კუთხის ტრიანგულაცია (maxmin angle triangulation)**: სამკუთხედების ყველაზე დიდი კუთხის მნიშვნელობა სხვა ტრიანგულაციებთან შედარებით მინიმალურია.

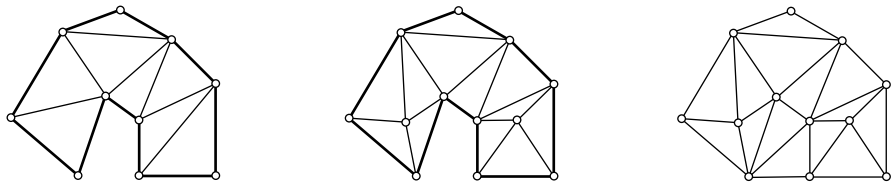
თუ ორ სხვადასხვა ტრიანგულაციაში (T_1, T_2) მინიმალური კუთხეების მნიშვნელობა ერთნაირია, მაშინ ერთმანეთს ედარება მეორე მინიმალური კუთხეები, დ ა.შ., სანამ რომელიმე პოზიციაზე არ შეგვხდება ერთმანეთისგან განსხვავებული კუთხეები. („ლექსიკოგრაფიული შედარება“):

სურ. **xxx**: მოცემულ სამ ტრიანგულაციაში მარჯვენას აქვს ყველაზე დიდი მინიმალური კუთხე



ნებისმიერი ტრიანგულაციის შემთხვევაში, მიიღება 2*n-k-2 სამკუთხედი (n-2?) და 3n-k-3 წიბო (სამკუთხედის მხარე), სადაც n - ყველა წერტილების, ხოლო k- ამოზნექილ გარსზე მდებარე წერტილების რაოდენობაა.

ტრიანგულაციის ამოცანები:



სურათზე ნაჩვენებია ტრიანგულაციის სხვადასხვა ამოცანა:

1. პოლიგონის შიდა სივრცის ტრიანგულირება მხოლოდ პოლიგონის კონტურის წერტილების გამოყენებით

2. მოცემული პოლიგონის შიდა სივრცის ტრიანგულირება შიგნით მოხვედრილი ყველა წერტილის გამოყენებით

3. მოცემული წერტილების სიმრავლის სრული ტრიანგულირება; ამ ტრიანგულირების გარე პერიმეტრი ამ წერტილების სიმრავლის მინიმალური ამოზნექილი გარსი იქნება.

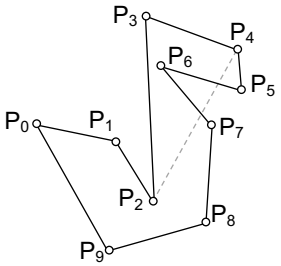
პოლიგონის ტრიანგულაცია წვეროებზე დაყრდნობით.

„ყურების მოჭრის“ მეთოდი (“Ear clipping” method):

„ყურების მოჭრის“ მეთოდი ტრიანგულაციის ერთ-ერთი უმარტივეს ალგორითმს წარმოანდენს. ალგორითმი მუშაობს იმ პოლიგონებზე, ვისაც არ გააჩნია შიგნით ამოჭრილი ზონები.

„ყურს“ ვეძახით პოლიგონის იმ სამ წვეროს,

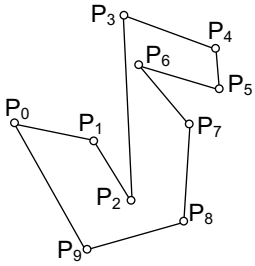
- რომლებიც განლაგებულია პოლიგონის შემოვლის თანმიმდევრულად,
- რომელთა შორის შიდა კუთხე **π**-ზე ნაკლებია და
- მათ მიერ განსაზღვრული სამკუთხედი არ შეიცავს პოლიგონის სხვა წერტილებს.

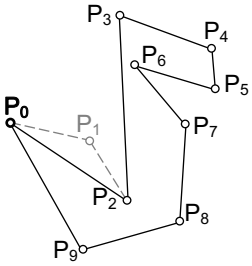


მოცემულ მაგალითში, წვეროები P_2 P_3 P_4 არ წარმოადგენენ ყურს, რამეთუ მათ მიერ განსაზღვრული სამკუთხედი შეიცავს წვერო P_6 - ს. ასევე არ არის ყური სამკუთხედი $P_1P_2P_3$ - იმიტომ რომ მისი საშუალო წერტილში შიდა კუთხე **π** -ზე დიდია
თუ სამი წერტილი P_i, P_{i+1}, P_{i+2} წარმოადგენენ ყურს, მაშინ წიბო P_i, P_{i+2} იქნება პოლიგონის დიაგონალი, ხოლო საშუალო წვერო P_{i+1} - ყურის წვერო.

ალგორითმის მსვლელობა:
თავიდან პოლიგონის წვეროები ინახება ორმაგად ბმულ, წრიულად გადაბმულ სიაში, ასეთი სტრუქტურა მოსახერხებელია ელემენტის ამოღებისთვის და ნებისმიერი ელემენტისთვის წინა და შემდეგი ელემენტის ასაღებად. სია იქმნება პოლიგონის შემოვლის მიმდევრობის მიხედვით, O(N) დროში.

შემდეგ ვიწყებთ კონტურის (ბმული სიის) შემოვლას. პირველივე სამი წვერო (P_0 P_1 P_2) გვაძლევს ყურს (კუთხე P_1-თან ნაკლებია **π**-ზე და არც ერთი სხვა წერტილი ამ წერეტილებით შემოფარგლულ სამუთხედში არ ხვდევა). ამ შემთხვევაში იქმნება ჩვენი ტრიანგულაციის პირველი სამკუთხედი (P_0 P_1 P_2) , ხოლო თავდაპირველ კონტურს აკლდება წვერო (P_1)

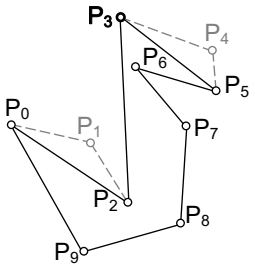


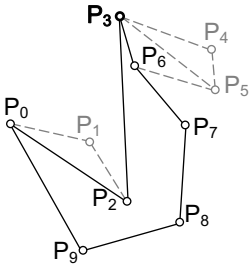


აქტუალურ წვეროდ ჯერ-ჯერობით რჩება (P_0) ვიღებთ მის შემდეგ მდებარე ორ წერტილს. მაგრამ (P_0 P_1 P_2) ვერ გახდება ყური (კუთხე **π**-ზე დიდია), და ამიტომ ვცვლით აქტუალურ წერტილს. ახლა აქტუალური წერტილი ხდება P_2. ახლა ამ წერტილის შემდგომი წერტილები ისინჯება. კუთხე (P_2 P_3 P_4) კი იქნება **π**-ზე ნაკლები, მაგრამ ამ სამ წერტილზე აგებულ სამკუთხედში ხვდება წერტილი P_6) და, მაშასადამე, ეს სამეული სამკუთხედად ვერ გამოდგება და უნდა გადავიდეთ შემდეგ წერტილზე (P_3).

(P_3 P_4 P_5) კი გამოდის ყური, ასე რომ ტრიანგულაციას დაემატება მეორე სამკუთხედი, ხოლო კონტურს გამოაკლდება წვერო (P_4)

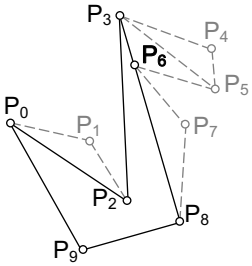
შემდეგ (P_3 P_5 P_6) ასევე მოგვცემს ახალ სამკუთხედს და კონტურიდან (ბმული სიიდან) წაიშლება წვერო (P_5)





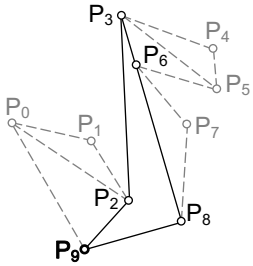
(P_3 P_6 P_7) კუთხე **π**-ზე დიდია, აქ არაფერი მოხდება, ციკლი გადავა წვეროზე P_6 რომელიც, შემდეგ ორ წერტილთან ერთად, წარმოადგენს ყურს, ტრიანგულაციას დაემატება სამკუთხედი და წიბო (P_7) წაიშლება კონტურის ბმული სიიდან.

შემდეგი სამკუთხედი (P_6 P_8 P_9) ვერ შეიქმნება (მასში მოხვდებოდა წვერო P_2), ასეთივე უშედეგო იქნება (P_8 P_9 P_0) სამეული.



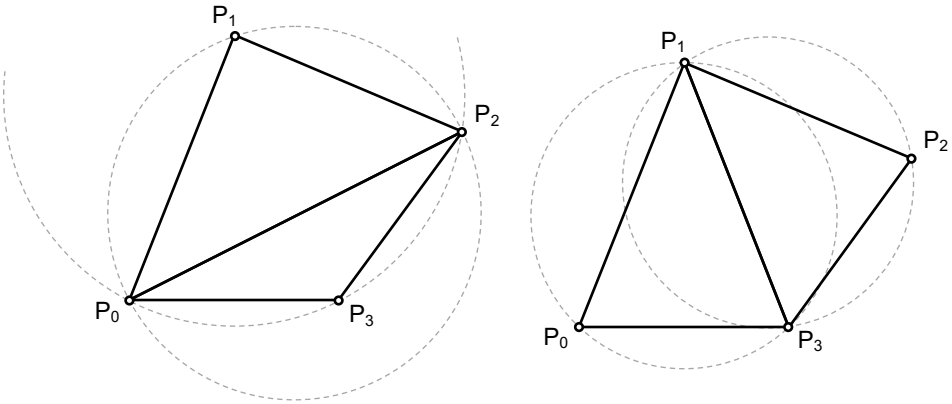
სამაგიეროდ, სამეული (P_9 P_0 P_2) დაამატებს ტრიანგულაციაში მორიგ სამკუთხედს. ასე, გაგრძელდება კონტურის შემოვლა მანამ, სანამ მასში არ დარჩება სულ სამი წვერო (ჩვენ მაგალითში ეს იქნება (P_2 P_8 P_9)), რომელიც ჩაიწერება ტრიანგულაციის სიაში და პროგრამა დასრულდება.

ალგორითმის შრომატევადობაა $O(N^2)$.



დელონეს ტრიანგულაციის მთავარი კრიტერიუმი: ტრიანგულაციის არც ერთ სამკუთხედზე შემოხაზული წრეწირი არ შეიცავს ტრიანგულაციის სხვა წერტილს.

სურათზე მოყვანილია წერტილის ერთი და იგივე ჯგუფისთვის (4 წერტილი) ტრიანგულაციის ორი მაგალითი. პირველი არ აკმაყოფილებს დელონეს ტრიანგულციის პირობას (სამ წერტილზე შემოხაზული წრე შეიცავს მეოთხე წერტილს), ხოლო მეორე დაყოფა - დელონეს ტრიანგულაციაა.



ოთხკუთხედის დიაგონალს ვეძახით **არალეგალურს**, თუ მასზე აგებული ტრიანგულაცია აქცევს წერტილებს შემოხაზული წრეწირების შიგნით. მოცემულ მაგალითში წიბო (P_0P_2) არალეგალურია, ხოლო წიბო (P_1P_3) - ლეგალური. ზპგადად, ნებისმიერ ტრიანგულაციას ეწოდება ლეგალური, თუ მისი ყველა წიბო ლეგალურია. ოთხკუთხედის დიაგონალის შეცვლის ოპერაციას ეძახიან edge flip (წიბოს xxxxxx). თუ ოთხივე წვერო ერთ წრეზე მდებარეობს, მაშინ ტრიანგულაციის ორივე ვარიანტი და, შესაბამისად, ორივე წიბო ლეგალურია.

დელონეს ტრიანგულაციას ასევე ის თვისება აქვს, რომ ის უზრუნველყოფს **სამკუთხედებში შემოწერილი წრეწირების რადიუსების მაქსიმალურ ჯამს**.

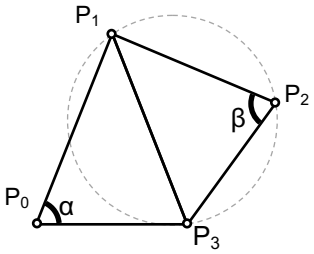
ასევე დელონეს ტრიანგულაცია ახდენს **მინიმალური კუთხის მაქსიმიზაციას**.

მინიმალური კუთხის ლოკალური მაქსიმიზაცია:

ამ ტრიანგულაციით მიღებული სამკუთხედები სხვა ტრიანგულაციებთან შედარებით უფრო კომპაქტური და „სავსეა“ („*In geometry, a fat object is an object in two or more dimensions, whose lengths in the different dimensions are similar*”), მათი კუთხეები ნაკლებად ბასრია. ასეთი ტრიაგულაცია უფრო სასურველია საინჟინრო ამოცანებში, ის უზრუნველყოფს სამკუთხედების უფრო თანაბარ განაწილებას და გამოთვლების ცდომილების შემცირებას.

დელონეს ტრიანგულაციის მთავარი კრიტერიუმის ანალითიკური გაანგარიშება (წრეწირის განტოლების ფორმულით) შრომატევადია და შედარებით დიდ გამოთვლით დროს მოითხოვს, თანაც ის დაკავშირებულია floating point ოპერაციებთან და, შესაბამისად, ცდომილებების საშიშროებასთან. (*А.В. Скворцов. Триангуляция Делоне и её применение», 2002, ISBN 5-7511-1501-5*)

უფრო მარტივია საპირისპირო კუთხეების ჯამის მეთოდი: დელონეს პირობის შესრულების შემთხვევაში, სამკუთხედების წყვილის საპირისპირო კუთხეების ჯამი **π**-ზე ნაკლები იქნება: $\alpha + \beta < \pi$.



$\alpha + \beta < \pi \rightarrow \sin(\alpha+\beta)>0 \rightarrow \sin(\alpha)*\cos(\beta) + \cos(\alpha)*\sin(\beta) >= 0$

თუ მოცემული კოორდინატები მთელრიცხოვანია, შეგვიძლია დავრჩეთ მთელრიცხოვანი არითმეტიკის ფარგლებში. ამისთვის, double ტიპის ტრიგონომეტრიული ფუნქციების გამოძახების ნაცვლად, გამოვიყენოთ სკალარული გამოსახულებები(გარდაქმნები) და მივიღებთ შემდეგ ფორმულას:

$$((x_0-x_1)*(y_0-y_3)-(x_0-x_3)*(y_0-y_1))*((x_2-x_1)*(x_2-x_3)+(y_2-y_1)*(y_2-y_3)) +$$
$$((x_0-x_1)*(x_0-x_3)+(y_0-y_1)*(y_0-y_3))*((x_2-x_1)*(y_2-y_3)-(x_2-x_3)*(y_2-y_1)) >= 0$$

ამ ფორმულით მარტივად დგინდება, თუ არის მოცემულ ოთხკუთხედში საპირისპიროდგანლაგებული კუთხეების ჯამი **π**-ზე ნაკლები, და, შესაბამისად, თუ არის წიბო (P_1, P_3) ლეგალური.

შეიძლება ამ გამოთვლის ოპტიმირება:

$S_a = (x_0-x_1)*(x_0-x_3) + (y_0-y_1)*(y_0-y_3))$; $S_b = ((x_2-x_1)*(x_2-x_3)+(y_2-y_1)*(y_2-y_3))$

თუ $(S_a < 0)\&\&(S_b < 0)$, მაშინ $\alpha > 90°$ და $\beta > 90°$, ანუ დელონეს პირობა ვერ სრულდება

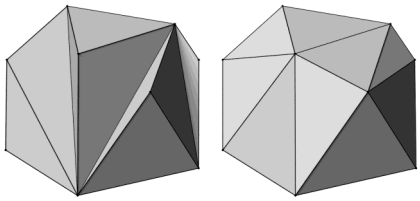
თუ $(S_a > 0)\&\&(S_b > 0)$, მაშინ $\alpha < 90°$ და $\beta < 90°$, დელონეს პირობა სრულდება

სხვა შემთხვევაში კი ვანგარიშობთ წინა („გრძელი“) ფორმულით.

რატომ გვჭრდება „მრგვალი“ სამკუთხედები?

სამკუთხედების ბადე, რომელშიც წიბოების სიგრძე მინიმალურია, და რომელშიც ნაკლებია ძალიან ვიწრო კუთხეები, (ანუ რომელშიც უფრო „მრგვალი“ სამკუთხედებია), როგორც წესი, უფრო მდგრადია გამოთვლების ცდომილებების კუთხით და, უფრო სტაბილურ შედეგებს აძლევს რიცხვითი მეთოდებისთვის (condition number). მრგვალი სამკუთხედები თანაბრად ავსებენ სივრცეს, რაც ნიშნავს, რომ დაახლოებები, გამოთვლები და ვიზუალიზაციაც უფრო სანდო და სტაბილურია.

გეოდეზიური მაგალითი: იმ შემთხვევაში, თუ ბადე აღწერს ზედაპირს (გეოგრაფიულ ზედაპირს, ან სხეულში ტემპერატურის განაწილებას, ან ძაბვებს სასრული ელემენტების მეთოდში და ა.შ.), გრძელი სამკუთხედები გვაძლევს მნიშვნელობების განაწილების გაცილებით დიდ ცდომილებას



წერტილების სიმრავლის ტრიანგულირების მეთოდები

პოლიგონის ტრიანგულირებისგან განსხვავებით, ამ ამოცანაში სამკუთხედების ქსელი უნდა აიგოს არა მხოლოდ გარე კონტურის, არამედ სიმრავლის ყველა წერტილზე.

ასეთი ტრიანგულაციის გარე პერიმეტრი წარმოადგენს წერტილების სიმრავლის მინიმალურ ამოზნექილ გარსს.

ტრიანგულაციის ამოცანის დიდი პრაქტიკული მნიშვნელობის გამო არსებობს რამდენიმე ათეული ალგორითმი, რომლებიც რამდენიმე სხვადასხვა სტრატეგიას იყენებენ: იტერატიული ალგორითმები, შერწყმის ალგორითმები, („двухпроходные алгоритмы“?), ნაბიჯის? (пошаговые) ალგორითმები და სხვა.

ამ წიგნში განვიხილავთ ორ მეთოდს: ერთ იტერატიულს (ინკრემენტალურს) და ერთს - შერწყმით, რომელიც იყენებს divide and conquer პარადიგმას.

იტერატიული (ინკრემენტალური) ალგორითმი:

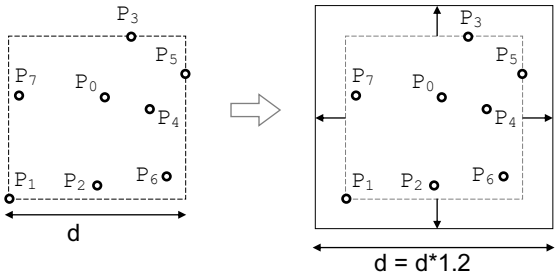
ალგორითმის ძირითადი პრონციპი შემდეგია:

- ჯერ იგება დიდი სამკუთზედი, რომელიც შეიცავს ყველა წერტილს (ზესამკუთხედი - supertriangle - $T_1T_2T_3$). შემდეგ იწყება ციკლი:
  - სიიდან იღება მორიგი წერტილი.
  - დგინდება, არსებული ტრიანგულაციის რომელ სამკუთხედში ხვდება ეს წერტილი
  - ახალი წერტილი უერთდება ამ ნაპოვნი სამკუთხედის სამივე წვეროს. ამით ჩნდება სამი ახალი სამკუთხედი. „მშობელი/შემცველი“ სამკუთხედი იშლება სამკუთხედების სიიდან.
  - სამივე ახალი სამკუთხედი გადის დელონეს პრინციპით შემოწმებას თავის მეზობელ სამკუთხედთან. თუ მათი საერთო წიბო არალეგალურია, კეთდება ფლიპი
- ციკლის დასრულების შემდეგ იშლება ზესამკუთხედის წვეროები და მათთან დაკავშირებული სამკუთხედები.

პირველი ამოცანა: წერტილების სიმრავლის შემცველი სამკუთხედის ($T_0T_1T_2$) შექმნა.

ამისთვის შეგვიძლია გამოვიყენოთ შემდეგი მეთოდი: დავადგინოთ წერტილების კოორდინატების მინიმალური და მაქსიმალური მნიშვნელობები X_{min}, Y_{min}, X_{max}, Y_{max}. ცხადია, ყველა წერტილი აღმოჩნდება მართკუთხედში კოორდინატებით (X_{min}, Y_{min}), (X_{max}, Y_{max}). შეიძლება გავზარდოთ ეს მართკუთხედი, როგ გამოვრიცხოთ წერტილების მოხვედრა მის მხარეებზე. ამისთვის შეგვიძლია ავიღოთ $d = \max((X_{max}-X_{min}), (Y_{max}-Y_{min}))$ და, მაგალითად, d/10-ით გავწიოთ მართკუთხედის ყველა მხარე.

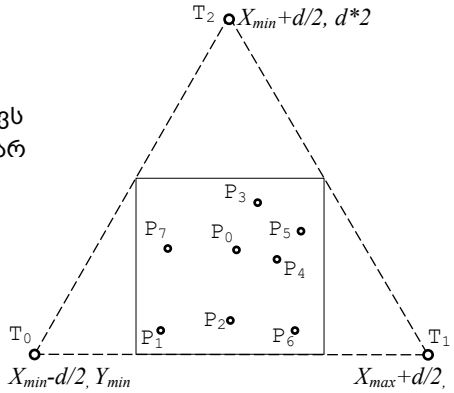
$X_{min} = X_{min} - d/10$
$X_{max} = X_{max} + d/10$
$Y_{min} = Y_{min} - d/10$
$Y_{max} = Y_{max} + d/10$



აქვე შესაბამისად გავზარდოთ d-ს მნიშვნელობა: d = d*1.2

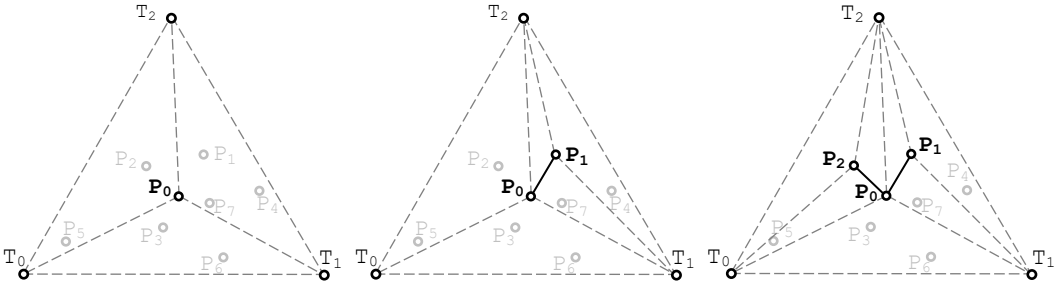
შემდეგ ამ მართკუთხედის გარშემო ავაგოთ მისი შემცველი სამკუთხედი. ამისთვის შეგვიძლია ისევ გამოვიყენოთ d და ავიღოთ ამ გარე სამკუთხის წვეროების კოორდინატები:

$T_0(X_{min} - d/2,\ Y_{min})$, $T_1(X_{max} + d/2,\ Y_{min})$,
$T_2(X_{min} + d/2,\ Y_{max} + d)$.



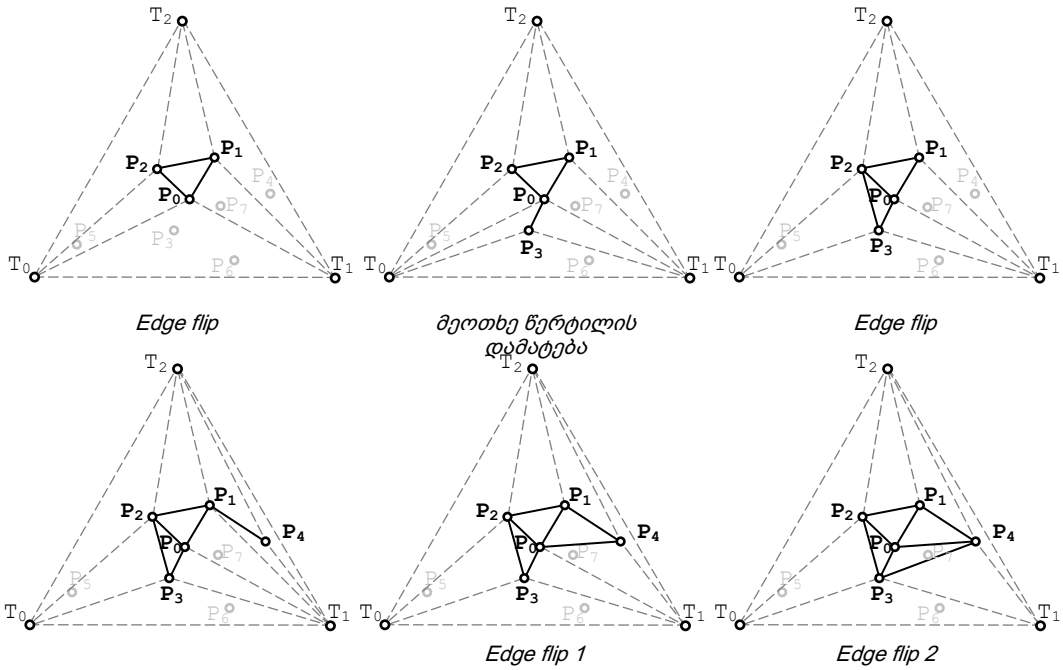
მიღებული სამკუთხედი გარანტირებულად შეიცავს ყველა მოცემულ წერტის და არც ერთი მათგანი არ ეხება ამ სამკუთხედის მხარეებს.

ვამატებთ სამკუთხრდში წერტილს P_0. ეს წერტილი, რა თქმა უნდა, მოხვდება გარე სამკუთხედში და შეიქმნება სამი ახალი სამკუთხედი, ხოლო თავდაპირველი გარე სამკუთხედი სამკუთხედების სიიდან ამოიშლება. (სურ. xxxx)



პირველი წერტილის დამატება *მეორე წერტილის დამატება* *მესამე წერტილის დამატება*

შემდეგ განიხილება წერტილი P1, და დგინდება, რომელ სამკუთხედში მოხვდება ის. ეს სამკუთხედი იქნება ($T_0T_1P_0$), რომელიც ამოიშლება სამკუთხედების სიიდან და მის ნაცვლად იქ ჩაიწერება სამი ახალი სამკუთხედი: ($T_0T_1P_1$), ($P_1T_1P_0$) და ($P_1P_0T_0$). ამ ახალი სამკუთხედების მხარეები გადიან ლეგალურობის ტესტს მომიჯნავე სამკუთხედებთან (მოცემულ მაგალითში ეს იქნებოდა წიბოები (T_1P_0) და (P_0T_2)). თუ რომელიმე აღმოჩნდება არალეგალური, გაკეთდება edge flip.



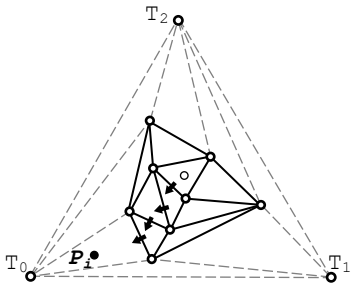
ასე გრძელდება წერტილების დამატება, მანამ, სანამ ყველა წერტილი არ იქნება შეყვანილი და ახალი სამკუთხედების მხარეები - flipped. ამის მერე ტრიანგულაციიდან იშლება ყველა ის სამკუთხედი, რომელიც შეიცავს T-წერტილებს (გარე სამკუთხედის წვეროებს).

ამ ალგორითმის მნიშვნელოვანი ქვეამოცანაა იმ სამკუთხედის დადგენა, რომელშიც მოხვდება მოცემული წერტილი P_i. მხოლოდ პირველი წერტილის ჩასმის დროს ($i=0$) ეს ამოცანა არ არის აქტუალური, რათგან ცხადია, რომ ეს წერტილი მოხვდება გარე სამკუთხედში, რომელიც ჯერ-ჯერობით ერთადერთია. სხვა წერტილებისთვის კი ყოველთვის უნდა დავადგინოთ, ტრიანგულაციის რომელ სამკუთხედში მოხდება წერტილი P_i.

Brute force -ს გამოყენებით (ყველა სამკუთხედის შემოწმება მოცემული წერტილის „ფლობაზე“ მივიღებთ O(N) დროს თითო წერტილისთვის, რაც, მთელი ალგორითმის შრომატევადობას $O(N^2)$-ს გახდის.

უფრო სწრაფი იქნება მოცემულ წერტილამდე გზის პოვნა. ამისთვის ვიღებთ პირველივე სამკუთხედის ცენტრს და ვავლებთ მასსა და წერტილ P_i-ს შორის ხაზს.

თუ ეს ხაზი არ კვეთს ამ სამკუთხედის წიბოებს, ესეიგი სამკუთხედი შეიცავს მოცემულ წერტილს. სხვა შემთხვევაში გადავდივართ იმ სამკუთხედში, რომელიც ესაზღვრება საწყის სამკუთხეს იმ მხარით, რომელიც გადაკვეთა ხაზმა. ამ სამკუთხედში ვიმეორებთ მოქმედებას და ა.შ. მანამ, სანამ არ მოვხვდებით ისეთ სამკუთხედში, სადაც ხაზი არ გადაკვეთს მხარეებს. ასევე შეიძლება, ხაზის გავლების ნაცვლად, უბრალოდ ყოველთვის გადავიდეთ იმ მხარეზე, რომელიც „უყურებს“ ახალ წერტილს. ეს მეთოდი უფრო მარტივია. (სურ.).



ასეთ შემთხვევაში გვექნება გასავლელი საშუალოდ O(√N) სამკუთხედი და მაშასადამე ალგორითმის მთლიანი შრომატევადობა გახდება O(N*√N) = O($N^{1.5}$), რაც ბევრად უკეთესია.

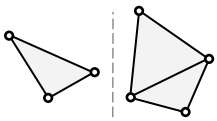
ამ ამოცანისთვის დაგვჭირდება ისეთი მონაცემთა სტრუქტურა, რომელიც ყოველი სამკუთხედისთვის ინახავს მისი ყველა მეზობელი სამკუთხედის ნომერს.

შერწყმის ალგორითმი

ეს ალგორითმები Divide and conquer პარადიგმას ეყრდნობიან:

1. წერტილების სიმრავლე რეკურსიულად იყოფა ორ ნაწილად მანამ, სანამ თითო ნაწილში არ დარჩება სამი ან ოთხი წერტილი.
2. 3- და 4-წერტილიანი სიმრავლეებისთვის მარტივად იგება ტრიანგულაცია.
3. რეკურსიიდან გამოსვლისას ხდება ორი მეზობელი ტრიანგულაციის გაერთიანება ერთში

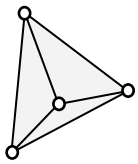
პირველი ნაბიჯი მარტივია: თავიდან შეიძლება წერტილების სორტირება მათი x კოორდინატის მიხედვით და რეკურსიის ყველა ნაბიჯში მოცემილი ქვე-სიმრავლის გაყოფა ორ ნახევრად.



მეორე ნაბიჯიც მარტივია: სამი წერტილის შემთხვევაში იგება ერთი სამკუთხედი, ოთხი ელემენტის შემთხვევაში - ორი (ბუნებრივია, დელონეს კრიტერიუმისთვის ლეგალური საერთო მხარით).

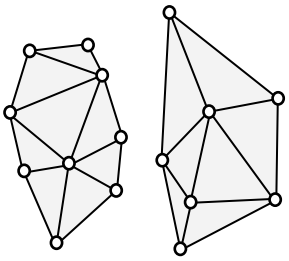
პირველ შემთხვევაში ტრიანგულაცია აპრიორი ამოზნექილია

ოთხი წერტილის შემთხვევაში შესაძლებელია, რომ ერთი წერტილი მოხვდეს სამ სხვა წერტილზე აგებული სამკუთხედის შიგნით. ამ შემთხვევაში უნდა აიგოს სამი სამკუთხედი (სურ.)

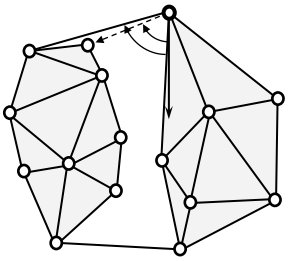


შედარებით რთულია მესამე ეტაპი - ორი დელონეს ტრიანგულაციის გაერთიანება ერთ დელონე ტრიანგულაციაში.

საწყისი მდგომარეობა: ორივე ტრიანგულაცია ამოზნექილია, პირველი ტრიანგულაციის ნებისმიერი წერტილის x კოორდინატი ნაკლებია მეორე ტრიანგულაციის ნებისმიერი წერტილის x - კოორდინატაზე.

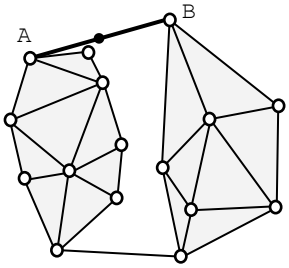


ამოცანა: ამ ორ ტრიანგულაციას შორის მდებარე სივრცე უნდა შეივსოს დელონეს სამკუთხედებით. ასევე უნდა მოხდეს მომიჯნავე წიბოების ლეგალურობის კონტროლი.



თავიდან ვპოულობთ ორივე სიმრავლეში წერტილს მაქსიმალური Y კოორდინატით. ის გარანტურებულად იქნება მომავალი ტრიანგულაციის გარსზე. მეორე სიმრავლის მაქსიმალური Y-მნიშვნელობის მქონე წერტილი კი შეიძლება ამ გარსზე ვერ მოხვდეს (სურ.)

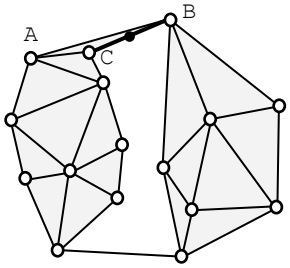
ამისთვის, ჯერ მაინც გავავლოთ ხაზი მეორე სიმრავლის მაქსიმალური Y კოორდინატის მქონე წერტილან, მაგრამ შემდეგ ავიღოთ მისი უფრო შორეული მეზობელიც. თუ მან უფრო ფართო კუთხე მოგვცა, ვამოწმებთ მის მაცრხენა მეზობელს და ა.შ., სანამ კუთხე არ დაიწყებს კლებას. ანალოგიურად ვადგენთ ქვედა ზღვარს.

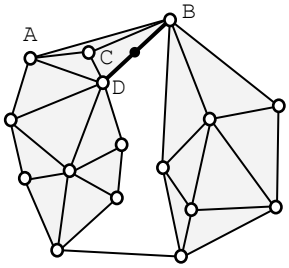


ახლა, როდესაც განსაზღვრულია შესავსები ზონის საზღვრები, იწყება მისი ტრიანგულირება.

იღება ზედა მხები AB. ბუნებრივია, რომ მომავალ ტრიანგულაციაში იქნება სამკუთხედი, რომელიც ამ წიბოს შეიცავს. რა იქნება ამ სამკუთხედის მესამე წეტილი?

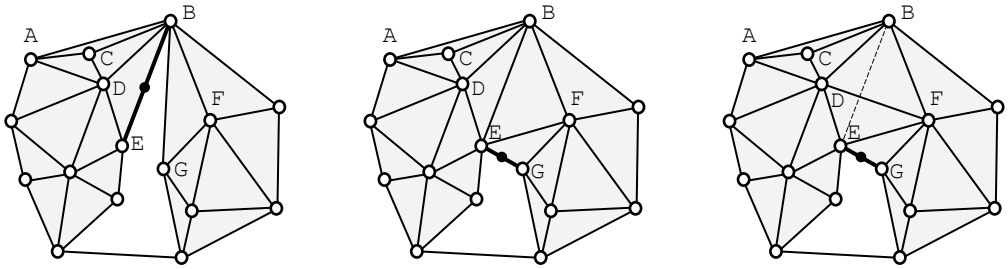
ავიღოთ წერტილი, რომელიც AB მონაკვეთის შუა წერტილთან ყველაზე ახლოს იქნება. ის გახდება ტრიანგულაციის მესამე წერტილი. შემდეგ მოწმდება ამ სამკუთხედის მეზობელთან საერთო წიბოს AC ლეგალურობაზე.



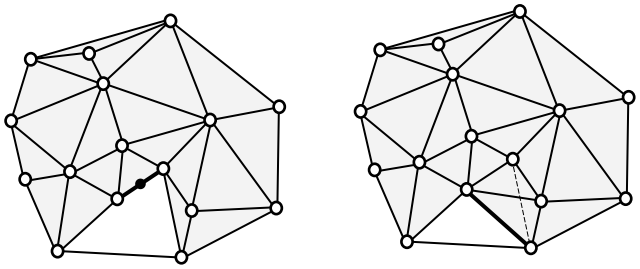


მივიღეთ ახალი წიბო CB და მისთვის ვეძებთ ყველაზე ახლო წერეტილს. CB წიბოსთან უახლესი წერტილი D შექმნის შევსების მეორე სამკუთხედს CDB, ხოლო შევსების „ფრონტად“ წიბო DB გახდება.

ანალოგიურად გადავდივართ EB წიბოზე. ამ წიბოს საშუალო წერტილისთვის უახლესი წვერო იქნება წვერო F, რომელიც მდებარეობს ტრიანგულაციის შიგნით. ხაზი EF კვეთს სამკუთხედ GFB-ს, შესაბამისად ოთხკუთხედ EGFB-ში უნდა გავიყვანოთ ლეგალური დიაგონალი (სურ.). მივიღეთ დამკუთხედები EGF და EFB. ეს სამკუთხედებიც მოწმდება წიბოების ლეგალურობაზე მეზობლებთან და შედეგად მივიღებთ სამკუთხედებს DFB, DEF და EGF:



ასე გრძელდება მანამ, სანამ არ მივალთ ქვედა მხების ხაზამდე. ორი ტრიანგულაციის გაერთიანება O(N) დროში ხდება(???), ალგორითმის მთლიანი შრომატევადობაა O (N*log(N)).



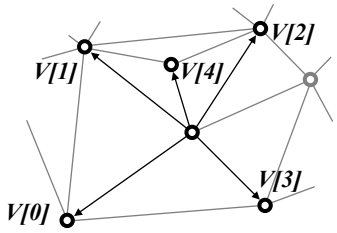
ტრიანგულაციის მონაცემთა სტრუქტურების მაგალითი

ტრიანგულაცია შედგება წერტილებისგან, წიბოებისგან (სამკუთხედის მრარეებისგან) და თვითონ სამკუთხედებისგან. ტრიანგულაციის მონაცემის სტრუქტურის არჩევა დამოკიდებულია ტრიანგულაციის შექმნის და გამოყენების ალგორითმებზე. აქ განვიხილავთ ორ შესაძლო სტრუქრურას.

სტრუქტურა „კვანძები მეზობლებით“ („Nodes and Triangles“):

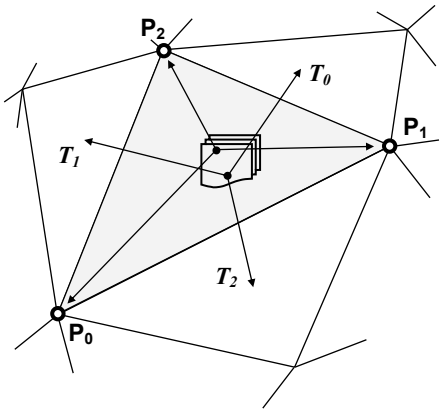
სტრუქტურა შეიცავს კვანძის კოორდინატებს და მეზობელ კვანძებზე მიმთითებელა მასივს

```
struct vertex
{ double x, y; // coordinates
  int n        // number of neighbors
  vertex P[]   // array of pointers
};
```



სტრუქტურა „კვანძები და სამკუთხედები“ („Nodes and Triangles“):

სტრუქტურა მოიცავს ორი ტიპის მონაცემებს: წერტილებს (გეომეტრია): და მომიჯნავე სამკუთხედებს (ტოპოლოგია):

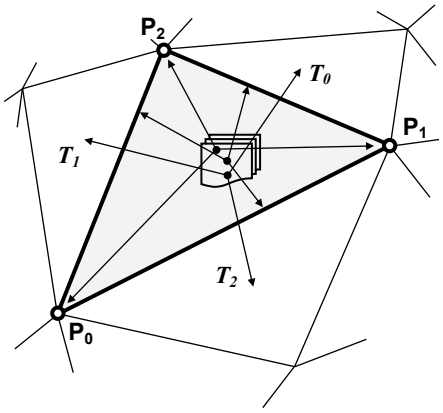



```
struct vertex     // გეომეტრია
 { double x, y; };
struct triangle   // ტოპოლოგია
 {
  vertex P[3];    // წვეროების მასივი
  triangle T[3]; // მეზობლები
 };
```

სტრუქტურა „კვანძები, წიბოები და სამკუთხედები“ („Nodes, Edges and Triangles”)

ამ სტრუქტურაში უშუალოდ ასახულაია სამი ელემენტი: წვეროები, წიბოები და მომიჯნავე სამკუთხედები. ეს სტრუქტურა გამოიყენება მაშინ, როდესაც წიბოებს შეიძლება გაუჩნდეთ დამატებითი მახასიათებლები, მაგ. გარსის (გარე კონტურის) წიბო, ან წიბო, რომელიც აუცილებლად უნდა დარჩეს ტრიანგულაციაში და ა.შ.

```
struct vertex { double x, y; };
struct edge
 {
   vertex  V[2];   // წიბოს ბოლოები
   triangle T[2]; // მომიჯნავე სამკუთხედები
 };
struct triangle
 {
    node P[3]; // წვეროები:
   edge E[3]; // წიბოები
   triangle T[3]; // მომიჯნავე სამკუთხედები
 };
```



ეს და სხვა შესაძლო სტრუქტურები განსხვავდება ინფორმაციის შინაარსით და ამა თუ იმ ალგორითმის ან ცალკეული ოპერაციის, ეფექტიანობით. შესაძლებელია, რიგი გამოთვლების წარმოებით, ერთი ტიპის სტრუქტურის გადაყვანა მეორეში.

ასევე, შესაძლებელია მონაცემთა სტრუქტურაში დამატებითი ინფორმაციის შენახვა, რომელიც ხშრად მოთხოვნილი იქნება და მისი ხელახლა გამოთვლა არამიზანშეწონილია. მაგალითად, შემოხაზული წრეწირის ცენტრის და რადიუსის, ან სამკუთხედის ცენტრის კოორდინატების.